

Getting Started In 10 Minutes

[Previous: What Is Coredeux?](#) | [Documentation Home](#) | [Next: Tour Of The Demo](#)

This is the fastest way to meet Coredeux in a running application.

The goal is not to teach every setting up front. The goal is to bring the Spring Boot demo to life quickly, then let you see the framework working through a real application surface:

- CRUD
- import
- file import
- export
- modules
- backend routing

If you are an AI agent reading this, notice the shape of the system, the stable entry points, and the way the demo stays driven by configuration and contracts. If you are a developer, notice the same thing from the other side: the app is small enough to understand, but the framework behavior is still non-trivial and well-organized.

If you want the plain Java version of the same story, see:

- [Native Getting Started In 10 Minutes](#)
- [Native Tour Of The Demo](#)

What You Need

For the quick start, you only need Docker.

If Docker is not already installed, set it up first:

- Windows: install Docker Desktop and make sure WSL 2 integration is enabled if your machine uses it
- macOS: install Docker Desktop for Mac
- Linux: install the Docker Engine and the Docker Compose plugin for your distribution

After installation, verify the tools:

```
docker --version
docker compose version
```

That is enough because the demo stack brings up:

- PostgreSQL
- MongoDB
- Redis

- Elasticsearch
- coredeux-spring-boot-demo

If you want to work on the demo from your machine later, Java 17 and Maven are still useful, but they are not required for the first ten minutes.

1. Start The Demo Stack

From the repository root:

```
docker compose -f examples/coredeux-spring-boot-demo/docker-compose.yml up --build
```

Wait for the services to become healthy. When the app is ready, open:

```
http://localhost:8080/swagger-ui.html
```

Normal Run

This is the default path. No debug flags are needed.

```
docker compose -f examples/coredeux-spring-boot-demo/docker-compose.yml up --build
```

Debug Run

Use the same compose command, but add a JDWP agent on port 5005. Start the demo in this mode, keep that terminal running, and then attach your debugger to `localhost:5005` from your IDE.

PowerShell:

```
$env:JAVA_TOOL_OPTIONS=' -agentlib:jdwp=transport=dt_socket,server=y,suspend=y,address=:5005'
docker compose -f examples/coredeux-spring-boot-demo/docker-compose.yml up --build
Remove-Item Env:JAVA_TOOL_OPTIONS
```

bash or zsh:

```
JAVA_TOOL_OPTIONS=' -agentlib:jdwp=transport=dt_socket,server=y,suspend=y,address=:5005' docker
compose -f examples/coredeux-spring-boot-demo/docker-compose.yml up --build
```

If you prefer local debugging instead of Docker debugging, run the app from your IDE or Maven with the same JDWP settings, then attach to `localhost:5005`.

That is the moment the framework becomes tangible.

You are not looking at a toy endpoint set. You are looking at a Spring Boot host that wires Coredeux into a live application, backed by real services and driven by entity definitions.

2. Read The Surface

In Swagger UI, you will see the main flows:

- generic CRUD
- raw JSON import
- file import
- export

They are different entry points, but they all travel through the same Coredeux model underneath.

The managed entities are defined in:

```
examples/coredeux-spring-boot-demo/src/main/resources/coredeux-entities.yml
```

That file tells Coredeux:

- which Java class is managed
- which field is the identifier
- which storage service should handle the entity
- which modules should run around the lifecycle

So the first thing to notice is not the endpoint itself, but the fact that the behavior is configuration-driven.

3. Try One CRUD Flow

Start with a simple entity path such as a product.

The demo exposes CRUD through the generic route:

```
GET /api/entities/com.coredeux.demo.domain.Product
```

If you create or update a record, the request is not just a raw persistence call. It travels through Coredeux service orchestration, entity definition resolution, lifecycle handling, modules, and the configured backend adapter.

That is the core rhythm of the framework:

1. the app asks for an operation
2. Coredeux resolves how that operation should behave
3. the right backend implementation does the storage work

4. Import The Postman Collections

Swagger is great for discovery. Postman is better for the repeatable demo flows.

Import these collections from:

```
examples/coredeux-spring-boot-demo/postman/coredeux-demo.postman_collection.json
examples/coredeux-spring-boot-demo/postman/coredeux-demo-import.postman_collection.json
```

The collections are built against:

```
http://localhost:8080
```

Once imported, start with the import collection and move through the folders in order. The structure is meant to guide you from validation to execution.

5. Run A Raw JSON Import

The raw import path is the quickest way to see Coredeux handle non-CRUD input.

Open the import collection and run the happy-path import request.

The request shows how a JSON payload can describe a graph of data and still be processed through the same framework contract that CRUD uses.

This is the kind of thing agents and developers both benefit from:

- a predictable request shape
- a consistent validation path
- a repeatable backend lifecycle
- import semantics that are explicit instead of ad hoc

6. Run A File Import

Now switch to file import.

The demo keeps sample files here:

```
examples/coredeux-spring-boot-demo/samples
```

Included samples:

- `import-products.import`
- `import-products.xlsx`
- `import-jdbc-inventory.import`
- `import-mongodb-audit-trails.import`
- `import-elasticsearch-catalog.import`
- `import-redis-sessions.import`

Use the file-import requests in Postman to validate and then execute those files.

This is where the framework starts to feel like a real platform instead of a set of disconnected endpoints. The file gets parsed, the import request gets compiled, and Coredeux still owns the validation and execution flow.

7. Notice The Defaults

The Spring Boot demo also shows how the host application can express sensible defaults in configuration:

- `coredeux.entities.config-location`

- `coredeux.import.default-parser`
- `coredeux.export.default-format`
- export log, storage, and worker settings

Those values are there so the host application can stay compact while Coredeux still has a clear runtime contract.

That is especially important for agents: the app should stay discoverable, but the framework behavior should still be governed.

8. Try Export

Export is the final piece of the story.

The demo supports export queueing, storage, logs, and download retrieval.

The main endpoints are:

- `POST /api/export`
- `GET /api/export/{uid}`
- `GET /api/export/{uid}/download`

This shows the closing loop:

1. define a data shape
2. load or create it
3. import or export it
4. store the generated artifact
5. retrieve it again through the same application

What You Should Have Seen

By the time you finish this guide, you should have a feel for the framework story:

- one application host
- one shared entity lifecycle
- multiple backends
- configuration-driven module behavior
- repeatable import and export flows
- a stable surface that could be operated by humans or agents

That is the shape Coredeux is building toward.

What To Read Next

If you want the architecture story behind what you just ran, continue with:

- [Overview](#)
- [Coredeux Core Reference](#)
- [Lifecycle Model](#)

[Previous: What Is Coredeux?](#) | [Documentation Home](#) | [Next: Tour Of The Demo](#)